

DeepCodeProbe: Towards Understanding What Models Trained on Code Learn

VAHID MAJDINASAB, Polytechnique Montreal, Canada

AMIN NIKANJAM, Polytechnique Montreal, Canada

FOUTSE KHOMH, Polytechnique Montreal, Canada

Machine Learning (ML) models trained on code and artifacts extracted from them (e.g., version control histories, code differences, etc.), provide invaluable assistance for software maintenance tasks. Despite their good performance, these models can make errors that are difficult to understand due to their large latent space and the complexity of interactions among their internal variables. The lack of interpretability in these models' decision-making processes raises concerns about their reliability, especially in safety-critical applications. Furthermore, the specific representations and features these models learn from their training data remain unclear, further contributing to the interpretability challenges and hesitancy in adopting these models in safety-critical contexts. To address these challenges and provide insights into what these models learn from code, we present DeepCodeProbe, a probing approach designed to investigate the syntax and representation learning capabilities of trained ML models aimed at software maintenance tasks. Applying DeepCodeProbe to state-of-the-art code clone detection, code summarization, and comment generation models provides empirical evidence that while small models capture abstract syntactic representations relevant to their tasks, their ability to learn the complete programming language syntax is limited. Our results show that increasing model capacity by increasing the number of parameters without changing the architecture improves syntax learning to an extent but introduces trade-offs in training and inference time alongside overfitting. DeepCodeProbe also uncovers specific code patterns and representations the models learn based on how code is represented for training the models. Our proposed probing approach uses a novel abstract syntax tree-based data representation that allows for probing models for syntax information. Leveraging our experimentation with the different models, we also provide a set of best practices for training models on code, to enhance both the models' performance and interpretability for code-related tasks. Additionally, our open-source replication package allows the application of DeepCodeProbe to interpret other code-related models. Our work addresses the critical need for reliable and trustworthy ML models in software maintenance. The insights from DeepCodeProbe can guide the development of more robust and interpretable models tailored for this domain.

CCS Concepts: **Software and its engineering** $\Rightarrow$ **Traceability**; *Software libraries and repositories; Software prototyping.*

Additional Key Words and Phrases: Code representation, deep learning, pre-trained language model, probing

ACM Reference Format:

Vahid Majdinasab, Amin Nikanjam, and Foutse Khomh. 2023. DeepCodeProbe: Towards Understanding What Models Trained on Code Learn. 1, 1 (July 2023), 32 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Authors' addresses: Vahid Majdinasab, vahid.majdinasab@polymtl.ca, Polytechnique Montreal, , Montreal, Quebec, Canada, ; Amin Nikanjam, Polytechnique Montreal, , Montreal, Quebec, Canada., amin.nikanjam@polymtl.ca; Foutse Khomh, Polytechnique Montreal, , Montreal, Quebec, Canada, , foutse.khomh@polymtl.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Association for Computing Machinery.

Manuscript submitted to ACM

Manuscript submitted to ACM

1 INTRODUCTION

Effective software maintenance is key to ensuring the long-term reliability and sustainability of software systems. Over time, growing codebases and rising code complexity pose challenges for maintaining software systems [30]. Machine learning (ML) models offer assistance in addressing software maintenance tasks such as code clone detection, fault localization, code smell detection, etc. [85]. Utilizing ML models trained on expansive code and related artifacts (e.g., statistics about the code such as the number of defined variables and function calls, documentation, control/data flow graphs, etc.) allows for facilitating various phases of software maintenance [85]. For example, data from the revision history of a project can be used for defect prediction, helping to address problems early on [5]. Moreover, leveraging Natural Language Processing (NLP) techniques alongside ML models has been shown to improve documentation interpretation, generation, and change assessment; reducing developers' time and mistakes [25]. Furthermore, Applying ML models for code clone detection helps to locate similar code inside the codebase, assisting developers in handling issues affecting multiple code copies [35].

Despite these benefits, however, there exist challenges concerning model interpretability [60] and the reliability of ML models in production [61]. Due to the black-box nature of state-of-the-art ML models, interpreting their decisions is difficult [9]. This lack of interpretability can lead to significant issues in software development and maintenance. For instance, when developers cannot interpret the decisions made by code models, debugging and repairing code becomes more complex and time-consuming [70, 93]. Misinterpreted model outputs can introduce subtle bugs that are hard to trace, potentially compromising software reliability and increasing maintenance costs [43, 52, 69]. Additionally, in collaborative environments, the inability to understand model decisions can hinder effective team communication and slow down the development process [51, 56, 79]. Therefore, understanding what ML models learn from their training data and how they generate outputs regarding the underlying task, is crucial for establishing their reliability and trustworthiness within safety-critical applications and production environments [4, 22]. Furthermore, examining the internals of ML models contributes to improving the overall performance by identifying decision-making errors that are amenable to fine-tuning or alternative architectural and data representation choices of models. One of the approaches used for this purpose is probing; in which a simple classifier is trained on the embeddings extracted from a model to predict a property of interest [29]. Probing ML models facilitates understanding the relationships between the models' input features and output predictions, allowing practitioners to assess whether the learned representations are aligned with expectations [92]. Given the recent rise of Large Language Models (LLM), researchers have been proposing novel approaches for probing LLMs for a variety of tasks and training data [12]. However, the majority of the proposed probing approaches are only applicable to transformer-based models with a large number of parameters. As such, there exists a gap in the current literature with a lack of probing techniques focusing on much smaller models trained on code for software maintenance tasks. To address this gap and given the benefits of having reliable models for software maintenance, we propose DeepCodeProbe, a probing approach designed for ML models trained on code for software maintenance tasks. DeepCodeProbe allows gaining insights into what ML models learn from their training data. Our probing approach is focused on identifying whether ML models used in software maintenance tasks learn the syntax of the programming languages that they have been trained on and the representations they learn from their inputs to achieve their objectives.

DeepCodeProbe is built upon the principles of AST-Probe [23], which examines LLMs' capability of encoding programming languages' syntax into their latent space. However in contrast to AST-Probe, DeepCodeProbe specifically

targets smaller ML models designed for software maintenance, distinct from large, transformer-based models. DeepCodeProbe is based on a novel data representation and embedding extraction approach for probing models trained on code. Leveraging DeepCodeProbe, we examine how models trained on code capture programming language syntax within their latent spaces and assess the effectiveness of our data representation and embedding extraction. Furthermore, we employ our probing approach to understand what representations of data the models under study learn from their training data to achieve their objectives. Finally, we formulate a set of best practices for training ML models on code for software maintenance tasks.

Briefly, this paper makes the following contributions:

- We propose a new probing approach for determining whether ML models trained on code learn the syntax of the programming language.
- We use DeepCodeProbe to investigate how ML models represent information from code in their latent space. We also investigate the effect of model size on their syntax learning capabilities.
- We formulate a set of best practices for training ML models for software maintenance.
- We publish all our data and scripts to allow for the replication and use of our approach at [42] for other researchers to use.

The rest of the paper is organized as follows: In Section 2, we present the background concepts and the related literature. We introduce DeepCodeProbe and our methodology alongside the details of our probing approach in Section 3. We describe our experiment design alongside the models under study in Section 4. We present our results in Section 5. We review the related works in Section 6, present the best practices for training small models on code in Section 7, and discuss threats to the validity of our study in Section 8. Finally, we conclude the paper in Section 9 and outline some avenues for future works.

2 BACKGROUND

In this section, we will review the necessary background and concepts related to our probing approach. DeepCodeProbe is designed to probe code models for software maintenance, therefore, we will first review the literature on ML for software maintenance. Afterward, we will discuss the probing approaches that have been proposed in NLP for ML models. Finally, as DeepCodeProbe is built on top of AST-Probe, we will describe AST-Probe and its underlying concepts.

2.1 Machine Learning for Software Maintenance

Software maintenance is a critical activity throughout the software development lifecycle. It involves continuous updates, testing, and improvements to ensure that software functions optimally according to evolving user requirements while preserving its original capabilities [55]. Given the increasing sophistication and size of contemporary software systems [30], their maintenance can become laborious, time-consuming, and error-prone [7, 45]. Consequently, numerous ML-based techniques have been proposed to simplify and enhance various aspects of software maintenance.

By training ML models on different software artifacts, such as source code [8, 21], version control history [5], documentation [19], and software metrics [31], these models can assist developers at various stages of software maintenance [85]. The primary applications of ML in software maintenance include:

- Defect prediction: ML models have been proposed to help developers anticipate potential problems in specific areas of the codebase. This enables efficient resource allocation and proactive issue resolution [13, 39, 71].

- Program repair: ML models have been proposed to help with automatically fixing common bugs. This can significantly reduce manual effort, minimize human errors, and save resources [8, 41, 74].
- Code clone detection: ML models have been proposed to detect clones in the codebase by identifying duplicate or similar segments of codes to facilitate refactoring efforts, ensuring consistency and reducing redundancy [17, 82].
- Code summarization: Proposed ML models allow for understanding complex code logic, easier maintainability, and collaboration by generating abstract representations of code blocks [1, 33].
- Comment generation: By producing descriptive comments, ML models allow for automatically increasing the intelligibility of code, ease of knowledge transfer, and support long-term maintenance activities [25, 37].

As our study is focused on ML models that are trained on code, we choose two representative tasks in software maintenance, namely code clone detection and code summarization, for probing and review the literature on these tasks, below.

2.1.1 Code Clone Detection. Code Clone Detection (CCD) is an important activity in software maintenance as a fault in one project can also be present in other projects with similar code. Identifying and correcting these clones is essential to ensure overall software quality. Tasks such as software refactoring, quality analysis, and code optimization often necessitate the identification of semantically and syntactically similar code segments [59]. In the CCD literature, clones are categorized into four types [58]:

- *Type-1:* Exact copies except for whitespace/comments.
- *Type-2:* Syntactically identical with different identifiers.
- *Type-3:* Similar fragments with added/removed statements.
- *Type-4:* Different syntax but similar semantics.

Detecting Type-1 and Type-2 clones is relatively straightforward, as they can be identified through syntactic comparisons. However, Type-3 and Type-4 clones pose a greater challenge, as they require capturing semantic similarities beyond surface-level syntax. Traditional approaches, such as text-based or token-based comparisons, often struggle with these more complex clone types [35]. To address this problem, the representation learning capabilities of Deep Learning (DL) models allow for the use of representations extracted from code such as Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), and software metrics for CCD. By utilizing such representations, DL-based approaches such as FCCA [26], CoCoNut [41], and Cclearner [36] have achieved high performance on CCD benchmarks.

2.1.2 Code Summarization and Comment Generation. Code summarization and comment generation are closely related tasks that aim to enhance code comprehension and maintainability. Code summarization involves generating concise natural language descriptions that capture the functional behavior or high-level semantics of a given code snippet [86]. On the other hand, comment generation focuses on producing comments that explain the purpose and functionality of specific code segments [65]. Several DL-based approaches have been proposed for code summarization and comment generation. Allamanis et al. [3] train a Convolutional Neural Network (CNN) with an attention mechanism for code summarization. This approach leverages the strengths of CNNs in capturing local patterns while the attention mechanism helps in focusing on the most relevant parts of the code for generating accurate summaries. Another work done by Hu et al. [25] uses the ASTs constructed from code snippets to train a sequence-to-sequence model for comment generation for Java methods. By encoding the structural information from the AST, their model can better capture the code's semantics and produce more relevant comments tailored to the specific code segments. Wei et al. [81] trained two models for code generation and code summarization by initializing a sequence-to-sequence model for each task and

training them alongside each other by using the loss of each network to improve the other. By effectively encoding the structural and semantic information from the code, these models can generate accurate and informative summaries and comments, facilitating code comprehension and maintainability.

2.2 Probing in Natural Language Processing

Probing is used in NLP as a methodology for assessing the linguistic capabilities learned by ML models [12]. In general, probing involves evaluating how well the model embeddings capture syntactic [24], semantic [84], or other types of linguistic properties inherent in language data [18, 53]. Probes are classifiers trained to predict specific features based on learned representations generated by trained models and a high classification accuracy indicates that the target feature is sufficiently encoded in those representations [76]. NLP probes can be categorized into several types, each focusing on different linguistic aspects [6, 40]:

- *Syntactic probes*: These probes assess whether the model representations capture syntactic properties of language, such as part-of-speech tags, constituent or dependency parsing structures, and word order information.
- *Semantic probes*: These probes aim to evaluate the model’s ability to encode semantic information, such as semantic roles, and lexical relations.
- *Coreference probes*: Coreference resolution is the task of identifying and linking mentions that refer to the same entity within a text [48]. Probing for coreference capabilities helps understanding if the model can capture long-range dependencies and resolve ambiguities.

By systematically probing for these different linguistic properties, researchers can gain insights into the strengths and weaknesses of various model architectures and investigate the specific representations learned by the models, increasing interpretability and reliability [6].

2.3 AST-Probe

AST-Probe is designed to explore the latent space of LLMs and identify the subspace that encapsulates the syntactic information of programming languages [23]. AST-Probe consists of the following steps: the AST of an input code snippet is constructed and transformed into a binary tree. Next, the binary tree is represented as a tuple of $\langle d, c, u \rangle$ vectors. It must be noted that the transformation from AST to binary tree, and from binary tree to $\langle d, c, u \rangle$ is bi-directional, meaning that by having the $\langle d, c, u \rangle$ tuple, one can reconstruct the AST of the input code. Since the AST constructed from a code snippet is syntactically valid (i.e., AST can only be constructed if code is syntactically correct), predicting the $\langle d, c, u \rangle$ tuple from the representations extracted from the hidden layers of the model, given a code snippet as input, indicates that the model is capable of representing the syntax of the programming language in its latent space. Figure 1b displays the AST extracted from a Python code snippet in Figure 1a. The $\langle d, c, u \rangle$ tuple is constructed by parsing the binary tree. In this tuple, d contains the structural information of the AST while c and u encode the labeling information. Algorithm 1 describes the construction of the $\langle d, c, u \rangle$ tuple in detail.

After constructing the $\langle d, c, u \rangle$ of an input code snippet I , the embeddings of the model M for the code are extracted from different layers. Suppose that model M has n layers: $l_1, \dots, l_n$. We denote the output of the l_{th} layer for input I as $M(I)_l$ with l being any layer between the second and the layer preceding the final layer (i.e., $2 \leq l \leq n-1$). After extracting $M(I)_l$ for an input code, the hidden representations are projected into a syntactic subspace S , and the probe is tasked with predicting the $\langle d, c, u \rangle$ from this projection. This process is repeated for all layers and all the code samples in the training data. If, for a given dataset of codes, the $\langle d, c, u \rangle$ tuples can be predicted by the probe with

Algorithm 1 Constructing the $\langle d, c, u \rangle$ tuple for AST-Probe[23]

```

1: procedure TREE2TUPLE(node)
2:   if node is leaf then
3:      $d \leftarrow []$ 
4:      $c \leftarrow []$ 
5:      $h \leftarrow 0$ 
6:     if node has unary label then
7:        $u \leftarrow [\text{node.unary\_label}]$ 
8:     else
9:        $u \leftarrow \theta$ 
10:    end if
11:  else
12:     $l, r \leftarrow \text{children of node}$ 
13:     $d_l, c_l, h_l \leftarrow \text{TREE2TUPLE}(l)$ 
14:     $d_r, c_r, h_r \leftarrow \text{TREE2TUPLE}(r)$ 
15:     $h \leftarrow \max(h_l, h_r) + 1$ 
16:     $d \leftarrow d + [h] + d_l + d_r$ 
17:     $c \leftarrow c + [\text{node.label}] + c_l + c_r$ 
18:     $u \leftarrow u + u_l + u_r$ 
19:  end if
20:  return  $d, c, u, h$ 
21: end procedure

```

high accuracy, it indicates that the model is capable of representing the syntax of the programming language in its latent space.

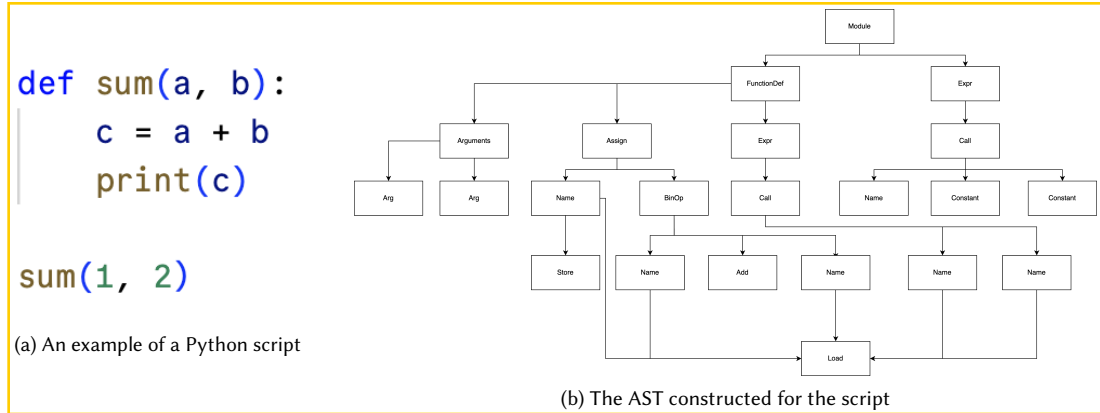


Fig. 1. An example of a Python script and its corresponding AST.

3 DEEPCODEPROBE

In this section, we first present DeepCodeProbe and how we represent AST/CFG of codes for probing the models. Afterward, we discuss our methodology for validating our proposed data representation and probing approach (i.e., DeepCodeProbe).

3.1 Feature Representation and Embedding Extraction

As described in Section 2.2, probing is used in NLP to gain an understanding of what the model learns from its training data and analyze how the model makes a decision based on each input [6]. Furthermore, probing can also be used as an interpretability approach to validate whether the model learns representations from the data required for solving the task at hand [12]. Previous works [23, 29, 72, 77] have explored the ability of large language models (LLMs) to encode syntactic information of programming languages within their latent spaces, using probing.

In this work, we are interested in DL models that are not as large as LLMs (i.e., language models trained on code that do not have millions/billions of parameters) and focus on smaller models that are trained on code for software maintenance tasks. Our proposed approach is probing such models on their capability of representing the syntax of the programming languages in their latent space. As such, we base our approach (DeepCodeProbe) on AST-Probe [23] as described in Section 2.3, to probe the models under study for syntactic information.

By basing our probing approach on the same principles of AST-Probe, we first extract a syntactically valid representation from the input code and construct a corresponding $\langle d, c, u \rangle$ tuple from it. However, unlike AST-Probe, in our approach, we do not derive the $\langle d, c, u \rangle$ tuple from the binary tree and instead build it directly from either the AST or the CFG extracted from the input code. We frame our probing approach as such since the binary tree of an AST or CFG would be very large and therefore, a $\langle d, c, u \rangle$ tuple constructed from it would be extremely high dimensional. Since we are interested in models that are much smaller than LLMs, probing these models for such high-dimensional representations will result in sparse representations of vectors (e.g., deriving a representation with 4,000+ features from an input with only 128 features). This sparsity would make the probe extremely sensitive to noise [34]. Given that the probe's capacity should be kept as small as possible (i.e. the probe should have significantly fewer parameters than the model being probed) [6, 23, 24], such sparse representations of vectors will result in non-convergence because of the accumulation of small effects of noise in high dimensional data [27, 75]. Therefore, we define the $\langle d, c, u \rangle$ tuple construction from the AST or CFG of a given code as follows:

- d : The position of each node in the AST/CFG in sequential order level by level (breadth-wise) from left to right, as shown in Figure 2.
- c : Position of the children of each node in sequential order for ASTs and the connections between each node in CFG.
- u : Vector representation of each node in the AST/CFG extracted from the model's data processing approach.

Algorithm 2 describes an overview of our probing approach. Similar to AST-Probe, extracting the $\langle d, c, u \rangle$ tuple from an AST/CFG is bi-directional, meaning that the $\langle d, c, u \rangle$ tuple extracted from the AST/CFG of a code can be used to reconstruct the corresponding AST/CFG. As this conversion is bi-directional, we infer that if the probe can predict the $\langle d, c, u \rangle$ tuple from the embedding extracted from a model, then the model is capable of representing the syntax of the programming language in its latent space. In the same manner, we consider low performance on predicting the $\langle d, c, u \rangle$ tuple, as the model's incapability to represent the syntax of the programming language in its latent space [23, 76].

```

1      d = [ 1, 2, 3, 4, 5, 6, ....., 22 ]
2      c = [
3          (2, 3) # children of node 1.
4          (4, 5, 6) # children of node 2,
5          (7) # children of node 3,
6          (8, 9) # children of node 4,
7          .....
8      ]
9      u = [
10         1 # Label of the ''Module'' node according to the model under study,
11         54, # Label of the ''FunctionDef'' node according to the model under
12             study,
13         32, # Label of the ''Expr'' node according to the model under study,
14         .....
15     ]

```

Listing 1. Generated $\langle d, c, u \rangle$ tuple for the AST in Figure ??**Algorithm 2** DeepCodeProbe

<pre> 1: procedure EXTRACTDCU(SCRIPT) 2: Input: SCRIPT 3: Output: DCU 4: if MODEL_INPUT_TYPE = AST then 5: SCRIPT_AST $\leftarrow$ ConstructAST(SCRIPT) 6: else if MODEL_INPUT_TYPE = CFG then 7: SCRIPT_CFG $\leftarrow$ ConstructCFG(SCRIPT) 8: end if 9: SCRIPT_DCU $\leftarrow$ Tree2Tuple(SCRIPT_AST or SCRIPT_CFG) 10: return SCRIPT_DCU 11: end procedure 12: procedure PROBE_MODEL(TRAINING_DATA, MODEL) 13: Input: TRAINING_DATA 14: Input: MODEL 15: Output: PROBING_RESULTS 16: Probe $\leftarrow$ InitializeProbe() 17: for code $\in$ TRAINING_DATA do 18: DCU $\leftarrow$ ExtractDCU(code) 19: model_embeddings $\leftarrow$ MODEL(code) 20: Predicted_DCU $\leftarrow$ Probe(model_embeddings) 21: accuracy_d, accuracy_c, accuracy_u $\leftarrow$ Compare(DCU, Predicted_DCU) 22: PROBING_RESULTS $\leftarrow$ PROBING_RESULTS $\cup$ {accuracy_d, accuracy_c, accuracy_u} 23: end for 24: return PROBING_RESULTS 25: end procedure </pre>	<pre> ▶ Extract DCU tuple from code ▶ The input code script ▶ The extracted DCU tuple ▶ If model uses ASTs ▶ Construct AST ▶ If model uses CFGs ▶ Construct CFG ▶ Convert AST/CFG to DCU tuple ▶ Probe trained model ▶ Code used to train MODEL ▶ Trained model ▶ Probing results ▶ Initialize probe ▶ Extract DCU tuple ▶ Get model embeddings ▶ Probe model ▶ Compare predictions ▶ Store results </pre>
---	--

Figure 2 displays the annotated AST that is used for $\langle d, c, u \rangle$ tuple construction (the code snippet is shown in Figure 1a and the constructed AST in Figure 1b). The red circles above each node indicate the assigned index with the

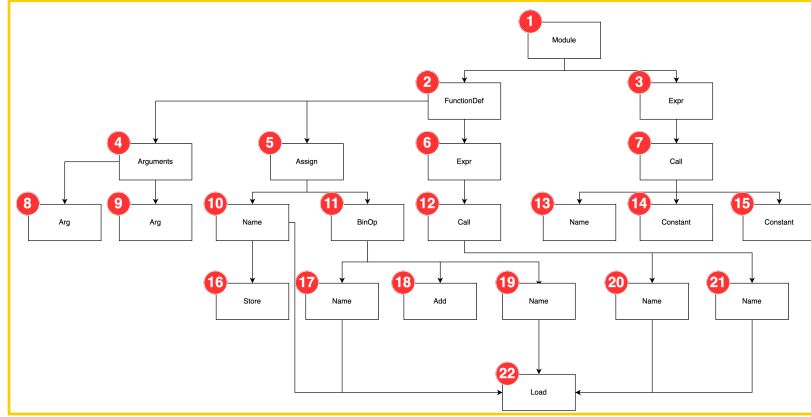


Fig. 2. Annotated AST

indexing being done level by level (breadth-wise) from left to right. Following this running example, Listing 1 displays the constructed $\langle d, c, u \rangle$ tuple which will be used for probing the model.

3.2 Validating DeepCodeProbe

To ensure the reliability and accuracy of our probing task, we validate our proposed approach through the following three stages:

- Validation of the data representation method: We validate that the $\langle d, c, u \rangle$ tuple constructed for each AST/CFG contains information on the programming language's syntax.
- Validation of the extracted embeddings: We validate that the embeddings extracted from the model contain enough *distinctive* information from the input. This is crucial to ensure that embeddings extracted from the model, for codes with dissimilar ASTs/CFGs are distinctly different from those extracted for codes with similar ASTs/CFGs.
- Validation that the embeddings contain information related to the task for which they are being probed: We validate that the models under study learn a relevant representation from their training data for the task for which they have been trained. This is crucial to ensure that the models under probe have not simply memorized their training data.

This validation is necessary because if the data representation approach or the extracted embeddings do not contain the information for which the model is being probed for, the results of the probing process will be meaningless since we would be probing the models for information that is not available in their latent space. Furthermore, if a model's embeddings do not change to reflect the task for which it has been trained (before and after training), these embeddings cannot be used for probing, since they would not contain enough information. In the following, we explain each of the aforementioned validation stages in more detail.

3.2.1 Validating data representation. DeepCodeProbe is designed for probing models that use syntactically valid constructs derived from code to accomplish their task. Previous works have shown that AST/CFG representations of code can be used to detect similarity between codes, as code clones of Types 1, 2, and weakly 3 have similar AST/CFG structures [2, 11, 66, 73]. Given this, we assume that the $\langle d, c, u \rangle$ tuples for codes that are exact or near clones

should be similar, while those for non-clones should be dissimilar. To validate this assumption, we construct $\langle d, c, u \rangle$ tuples from the AST/CFG of code clones and non-clone pairs and measure the cosine similarity between these tuples. Similarity is defined as the relative distance between clone and non-clone pairs in the $\langle d, c, u \rangle$ vector space. Since DeepCodeProbe works with vectors derived from tree and graph representations, using the cosine similarity metric allows us to assess the degree of similarity and dissimilarity between these vectors effectively. If the $\langle d, c, u \rangle$ tuples for similar codes (clones of Type 1 and 2) are significantly more similar to each other compared to those for dissimilar codes (clones of Type 3 and 4), we can conclude that the data representation is valid.

3.2.2 Validating the extracted embeddings. Previous works have shown that semantic similarity between data points is reflected in embedding proximity in DL models (i.e. if two inputs are similar, then their representations in the model’s latent space are similar) [14, 64]. Therefore, in order to validate the extracted embeddings we follow a similar approach to validating the data representation. In this step, instead of comparing the similarity of $\langle d, c, u \rangle$ tuples, we compare the similarity of the extracted embeddings. We use the same code clone tuples constructed for validating the data representation. As such, we input code clone and non-clone pairs to each model and extract the corresponding embeddings for each input. Afterward, we use cosine similarity to measure the distance of the model’s embeddings between code clones and non-clones. Given that the models under study are trained on syntactically valid representations of code, then regardless of the task that they are trained for, we expect that the extracted embeddings for code tuples that are syntactically similar (Types 1 and 2) to be similar to each other compared to code tuples that are not (Types 3 and 4). In other words, the cosine similarity for the embedding of similar codes should be significantly higher than the cosine similarity of dissimilar codes for the extracted embeddings to be valid.

3.2.3 Validating the embedded information. In probing literature, it is standard to show that the probe itself is incapable of learning representations from the extracted embeddings and instead only exposes information that already exists within the model’s latent space [23, 24, 77]. This is done by training the probe on embeddings extracted from randomly initialized versions of the model (i.e. models before training) and embeddings extracted from the model after it has been trained. If there exists a significant difference in the probe’s performance in predicting the property of interest, then it is assumed that the probe is incapable of learning representations from the embeddings itself [6, 23, 40, 76]. In our context, where we are probing small models trained on syntactic representations of code, we need to ensure that the probe’s validity does not rely on the models’ capacity to learn syntax. To address this, we propose an alternative validation method focusing on the similarity of embeddings for code clones, regardless of the models’ syntax learning capacity. Given that the models are trained on syntactic representations of code, we extract embeddings for code clone tuples from both the trained and randomly initialized versions of the models. We then compare the cosine similarity scores for code clones from the trained model against those from the randomly initialized model. Higher similarity scores for the trained model would suggest that the model has learned some form of meaningful representations, even if not explicitly syntactic. Next, we train the probe on these embeddings and assess its performance. A significant difference in the probe’s performance between embeddings from the trained and randomly initialized models would indicate that the probe is uncovering the existing structure in the embeddings rather than learning it during the probing process.

This comprehensive validation methodology ensures that our probe operates on reliable data representations and embeddings. By validating the data representation, we confirm that our $\langle d, c, u \rangle$ tuples accurately capture the syntactic structures of codes. Through validating the extracted embeddings, we ensure that the models reflect meaningful semantic similarities. Finally, by validating the probe, we establish that it exposes the pre-existing information within the model’s

latent space without learning representations itself. These steps collectively validate the data representation and the probing approach, allowing us to analyze and draw conclusions about the models' capabilities in learning syntactic properties of code.

4 EXPERIMENTAL DESIGN

In this Section, we will describe our experimental design. To conduct our study, we define the following research questions:

- **RQ1:** Are models trained on syntactical representations of code capable of learning the syntax of the programming language?
- **RQ2:** In cases where models trained on syntactical representations of code fail to learn the programming language's syntax, do they learn abstract patterns based on syntax?
- **RQ3:** Does increasing the capacity of models without changing their architecture improve syntax learning capabilities?

In the rest of this section, we will first describe the models we have chosen for our study. Afterward, we describe the approaches used to construct the dataset for probing each model and lay out the details of the probes used for the models under study for answering RQ1 and RQ2. Finally, we describe how we increase the capacity of the models under study for answering RQ3.

4.1 Models Under Study

In this study, we are interested in models trained on code for software maintenance tasks. As such, to answer our RQs, we select two tasks in software maintenance for which learning the syntax of the programming language is considered to be important: Code Clone Detection (CCD) and code summarization. For each of these tasks, we select two models based on the following criteria:

- They are not large language models (LLMs).
- They are trained on syntactically valid representations of code, such as Abstract Syntax Trees (ASTs) or Control Flow Graphs (CFGs), rather than raw code or code artifacts as text.
- They demonstrate high performance on benchmarks specifically designed for the tasks they have been trained for.

Considering the criteria defined above, we select the following models to assess our probing approach, DeepCodeProbe:

4.1.1 AST-NN. Zhang et. al.[87] proposed AST-NN as a novel approach for representing the AST extracted from code as inputs for DL models. In their approach, they break down the AST into Statement Trees (ST) to address the issues of token limitations and long-term dependencies in DL models, which arise from the large size of ASTs. This process involves a preorder traversal of the AST to identify and separate statement nodes defined by the programming language. For nested statements, statement headers and included statements are split to form individual statement trees. These trees, which may have multiple children (i.e., multi-way trees), represent the lexical and syntactical structure of individual statements, excluding sub-statements that belong to nested structures within the statement. This process reduces an AST into multiple smaller STs. Additionally, AST-NN employs a Word2Vec model [47], to encode the labeling information for each node in the AST by learning vector representations of words from their context in a corpus.

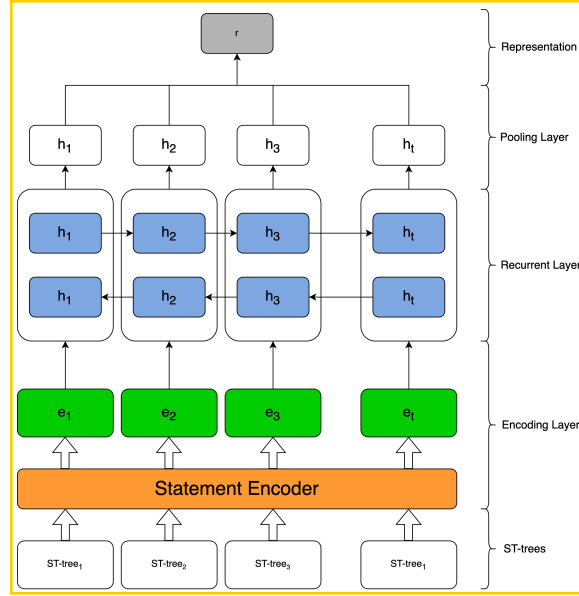


Fig. 3. Model architecture for AST-NN [87]

In AST-NN, the input code is broken down into STs to be used for two tasks, namely, source code classification and CCD. In this task, the STs extracted from two pieces of code are used as inputs for a DNN comprising an encoding layer, a recurrent layer, and a pooling layer. Figure 3 provides an overview of the AST-NN model architecture. The output of the representation layer is a binary value indicating whether two input codes are clones. The authors of AST-NN reported state-of-the-art clone detection results on the BigCloneBench [67] and OJClone [49] datasets for C and Java programming languages using their proposed approach.

4.1.2 FuncGNN. Similar to AST-NN, Nair et. al. [50] proposed FuncGNN, a graph neural network that predicts the Graph Edit Distance (GED) between the CFGs of two code snippets to identify clone pairs. FuncGNN, as presented in Figure 4, operates by extracting CFGs from input code snippets and utilizing a statement-level tokenization approach, which distinguishes it from the word-level tokenization used by other models under study, such as AST-NN, SummarizationTF, and CodeSumDRL. The core assumption of FuncGNN is that code clone pairs will exhibit a lower GED compared to non-clone pairs, leveraging this metric to determine whether two code samples are clones.

FuncGNN’s methodology combines a top-down approach for embedding the global graph information with a bottom-up one for atomic-level node comparison to catch similarities between operations. The top-down approach generates an embedding that captures the overall structure and control flow of a CFG by incorporating an attention mechanism to find nodes in the CFG that have high semantic similarity. On the other hand, the bottom-up approach, focuses on the comparison of node-level similarities within the CFGs, to capture the similarities in program operations. By incorporating both approaches, FuncGNN predicts the GED between two input CFGs.

4.1.3 SummarizationTF. The approach proposed by Shido et al.[63] extends the Tree-LSTM model [68] to handle source code summarization more effectively by introducing the Multi-way Tree-LSTM. This extension allows the model to process ASTs that have nodes with an arbitrary number of ordered children, which is a common feature in source

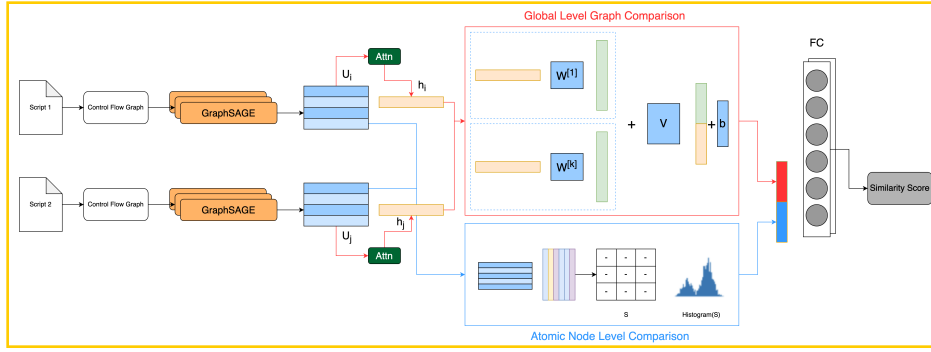


Fig. 4. Model architecture for FuncGNN [50]

code. In this approach, the AST of an input code is extracted and then each node in the AST is represented by a vector of fixed dimensions. These vectors are then used as input to the Multi-way Tree-LSTM and an LSTM decoder is then used to generate natural language summaries. Figure 5 presents an overview of SummarizationTF's operation.

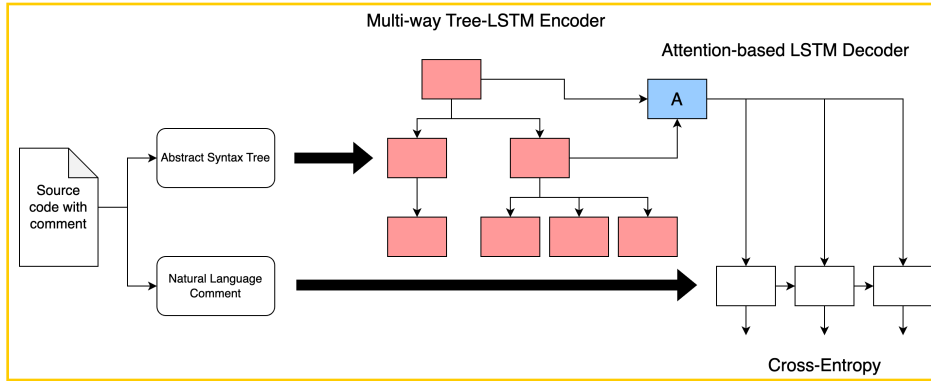


Fig. 5. Model architecture for SummarizationTF [63]

4.1.4 CodeSumDRL. Wan et. al.[78] proposed an approach for source code summarization based on Deep Reinforcement Learning (DRL). Their proposed approach integrates the AST structure and sequential content of code snippets as inputs to an actor-critic architecture as presented in Figure 6. The actor-network suggests the next best word to summarize the code based on the current state, providing local guidance, while the critic-network assesses the possibility of occurrence of all possible next states. Initially, both networks are pre-trained using supervised learning (i.e., code and documentation tuples), using cross-entropy and mean square as loss functions, for the actor and the critic respectively. Afterward, both networks are further trained through policy gradient methods using the BLEU metric as the advantage reward to improve the model's performance. To train the model, code is represented in a hybrid manner in two levels: lexical level (code comments) and syntactic level (code ASTs) [78]. An LSTM is used to convert the natural language comments and a separate Tree-based-RNN is used to represent the code ASTs. The output of these two layers is then used to represent the input code alongside its comment for training the actor and critic.

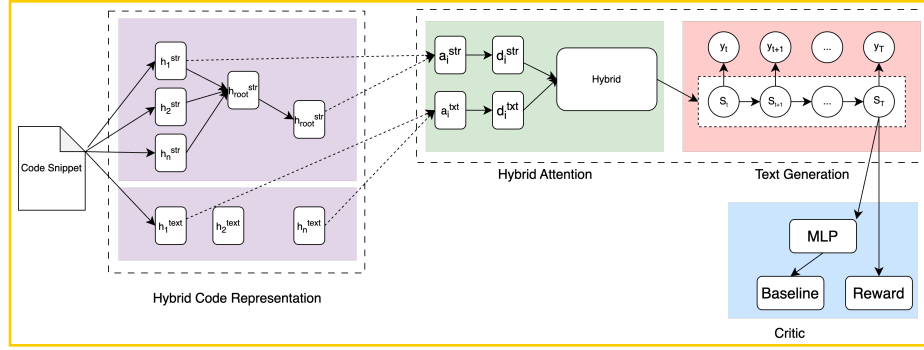


Fig. 6. Model architecture for CodeSumDRL [78]

For each of the models described above we use the code and data in the replication packages provided by the authors and train the models from scratch to replicate the results of the original studies [50, 63, 78, 87]. It should be noted that for each model, we do not modify the codes or the data in the replication packages and in cases where it was required (e.g., deprecated libraries, syntax errors in the code, etc.) we kept our modifications as minimal as possible. We provide the source code of each model alongside our probing approach in our replication package [42].

4.2 Dataset Construction for Each Model

After training models and replicating the results of the original studies, we use the same data that was used in the original works as input to the models and at the same time extract the AST/CFG from each input and construct the $\langle d, c, u, \rangle$ tuple which was explained in Section 3.1. Afterward, we extract the intermediate outputs of the model for the given input and train the probe to predict the constructed $\langle d, c, u, \rangle$ tuple from the extracted embeddings. It is important to note that the chosen models contain multiple hidden layers. Therefore, for each model, we extract the embeddings from the layers that provide the most information-rich representations learned by the model, as follows:

- **AST-NN:** The AST-NN model architecture, as shown in Figure 3, consists of three main components: an encoding layer, a recurrent layer, and a pooling layer. The encoding layer first converts the input STs as described in Section 4.1.1 into numerical representations suitable for the model. These encoded inputs are then processed by the recurrent layer. The outputs of the recurrent layer are subsequently passed through the pooling layer, which aggregates the representations and produces a fixed-size vector encoding of the entire input STs. For probing, we extract the embeddings from both the recurrent and the pooling layers.
- **FuncGNN:** The FuncGNN model architecture, as illustrated in Figure 4, consists of several components: a graph convolution layer, an attention mask mechanism, and a ReLU activation function. The graph convolution layer learns representations by capturing the structural information from the input CFGs. These learned representations are then passed through the attention mask, which assigns importance weights to different parts of the outputs of the graph convolution layer. Finally, a ReLU activation function is applied to the attention-weighted representations, and a sigmoid activation yields a binary output indicating whether the two input CFGs are code clones or not. For probing, we extract the embeddings from the output of the graph convolution layer.
- **SummarizationTF:** As displayed in Figure 5, the SummarizationTF model follows an encoder-decoder architecture. The encoder component processes the AST generated from a code snippet and generates a contextualized

representation from it. The outputs of the encoder layer are used as input to the decoder layer to generate summaries from the input AST. For probing, we extract the embeddings from the outputs of the encoder component.

- **CodeSumDRL:** The CodeSumDRL model employs a dual-encoder architecture as displayed in Figure 6, where separate LSTM layers are used to encode the input code snippet and the associated comments. The outputs of these LSTM layers are then passed as inputs to an encoder layer. For probing, we extract the embeddings from the outputs of the encoder layer.

The rationale behind selecting these specific layers is that they are expected to capture the most informative and high-level representations of the input data, as they are the final representations before subsequent transformations or output generations. After extracting the embeddings, the dataset used for training DeepCodeProbe for each model consists of (embeddings, $\langle d, c, u \rangle$) tuples from the inputs given to each model. We outline the detailed dataset construction for each of the models under study as follows.

4.2.1 AST-NN. Algorithm 3 presents how we construct the $\langle d, c, u \rangle$ tuples for probing AST-NN using DeepCodeProbe. Note that to represent the labels of each node, we use the same Word2Vec model that is used by AST-NN to convert the labels into numerical representations (i.e., indexes) as described in Section 4.1.1. We probe this model on its capability to represent the syntax of both C and Java programming languages. We also adopt the same preprocessing approach as AST-NN to convert the inputs from code to STs. Afterward, for each ST, we construct the $\langle d, c, u \rangle$ tuple to be used for training the probe.

4.2.2 FuncGNN. Algorithm 4 presents an overview of the $\langle d, c, u \rangle$ tuple construction for FuncGNN in DeepCodeProbe. The training data for FuncGNN is already pre-processed from code into CFGs using Soot¹, a framework that is used to transform Java bytecode into intermediate representation for analysis, visualization, and optimization. In this format, each node represents a line in the program, and the edges between the nodes indicate the control flow between each node. For constructing the $\langle d, c, u \rangle$ tuple, we traverse the CFG and use the depth, connections, and labels of each node to represent it in a numerical vector for training and evaluating the probe.

4.2.3 SummarizationTF. This model works with ASTs. Therefore, unlike AST-NN there is no need to first generate the ASTs from code and then break them down into STs. As such, the process for generating the $\langle d, c, u \rangle$ tuples is similar to that of Algorithm 3 with some modifications to the method of parsing the trees. Algorithm 5 presents our approach in detail.

4.2.4 CodeSumDRL. Similar to SummarizationTF, CodeSumDRL also works with ASTs generated from code. As such, we follow the same steps as outlined in Algorithm 5 to build the necessary $\langle d, c, u \rangle$ tuples for training and evaluating the probe.

4.3 Probe’s Design for Each Model

As explained in Section 2.2, the probe should have minimal size and complexity. Following the same conventions used in probing language models [44], the probe should not learn any information from the data but instead, learn a transformation (i.e., projection) of the embeddings that exposes the information that the model learns from its training data. Therefore, for each of the models under study, based on their capacity (i.e., number of parameters), we experiment

¹<https://github.com/Sable/soot>

Algorithm 3 Generating $\langle d, c, u \rangle$ Tuples from STs**Require:** An Abstract Syntax Tree (AST) *tree***Ensure:** Dictionary containing depth (D), child indices (C), and unary label (U) tuples for ST blocks

```

1: procedure TREE2TUPLE(tree)
2:   word2VecEmbeddings  $\leftarrow$  Load appropriate Word2Vec model based on language
3:   maxTokenIndex  $\leftarrow$  Number of vectors in word2VecEmbeddings
4:   vocab  $\leftarrow$  word2VecEmbeddings
5:   function CONVERTNODETOINDEX(node)
6:     tokenIndex  $\leftarrow$  Index of node.token in vocab or maxTokenIndex if not found
7:     childIndices  $\leftarrow$  Empty list
8:     for child in node.children do
9:       childIndices.append(CONVERTNODETOINDEX(child))
10:    end for
11:    return [tokenIndex] + childIndices
12:  end function
13:  function TREE2INDEX(rootNode)
14:    blocks  $\leftarrow$  Extract ST blocks from AST starting from rootNode
15:    sequenceInfo  $\leftarrow$  Initialize empty dictionary for block info
16:    for i = 0 to length(blocks) - 1 do
17:      block  $\leftarrow$  blocks[i]
18:      blockTokenIndex  $\leftarrow$  Index of block.token in vocab or maxTokenIndex
19:      blockChildrenIndices  $\leftarrow$  CONVERTNODETOINDEX(block)
20:      sequenceInfo[i].d  $\leftarrow$  i + 1
21:      sequenceInfo[i].c  $\leftarrow$  Flatten blockChildrenIndices excluding the first element
22:      sequenceInfo[i].u  $\leftarrow$  blockTokenIndex
23:    end for
24:    D  $\leftarrow$  Extract d values from sequenceInfo for all blocks
25:    C  $\leftarrow$  Extract c values from sequenceInfo for all blocks
26:    U  $\leftarrow$  Extract u values from sequenceInfo for all blocks
27:    return {d : D, c : C, u : U}
28:  end function
29:  return TREE2INDEX(tree.root)
30: end procedure

```

with probes with different sizes. To prevent the probes from learning representations from the model's embeddings, we follow the same design approach as used in [23]. This means that each probe has a single hidden layer, and the number of units in this layer is adjusted based on the size and architecture of the model being probed. We include the code for training probes of different sizes for each of the models under study in our replication package [42].

4.4 Scaling Up The Models

Kaplan et al. [28] examined the scaling laws for language models, focusing on the impact of increasing model capacity, training data, and computational resources on the models' training efficiency and performance. In their study, they experiment with how increasing each and a combination of these factors affects the model's sample efficiency, overfitting, convergence, and overall performance on the test benchmarks. Their research focuses on transformer-based language models and is therefore not directly applicable to our work. However, we can apply the same principles to the models under study and investigate how increasing the models' capacity affects their syntax learning capabilities. Based on

Algorithm 4 Generating $\langle d, c, u \rangle$ Tuples from CFGs**Require:** Code graphs represented as lists of edges and node labels**Ensure:** Tuples representing depth (D), connections (C), and unary label (U) for the code graphs

```

1: function CFG2TUPLE(codeGraphs, nodeLabelMappings)
2:   for codeGraph in codeGraphs do
3:     edges  $\leftarrow$  codeGraph.edges
4:     reverseEdges  $\leftarrow$  list of edges with source and target swapped
5:     edgeList  $\leftarrow$  edges + reverseEdges ▷ Ensure connections in both directions
6:     Convert edgeList to an appropriate data structure
7:     depths  $\leftarrow$  list of node depths in the graph traversal order
8:     connections  $\leftarrow$  edgeList
9:     usages  $\leftarrow$  list of node label mappings from nodeLabelMappings
10:    Store (depths, connections, usages) as a tuple for the current codeGraph
11:  end for
12:  return list of (depths, connections, usages) tuples for all code graphs
13: end function

```

their work, to address RQ3, we keep the amount of compute and the dataset size the same, while increasing the number of models' parameters (i.e., capacity) without changing the models' architecture. We have described the architecture of each of the models under study alongside the layers from which we have extracted the embeddings in Sections 4.1 and 4.2, respectively. To align our approach with the principles outlined by Kaplan et al. [28], we define "scaling up" for each model in our study as increasing model capacity by expanding the number of units within the layers responsible for generating outputs used for probing. Specifically, our scaling strategy includes:

- AST-NN: Increasing the recurrent and pooling layers' unit count and leaving the encoding layer unchanged.
- FuncGNN: We increase the number of units of the graph convolution layer without altering the number of attention heads.
- SummarizationTF: The Multi-way Tree-LSTM's unit count is increased, while all other components remain the same.
- CodeSumDRL: The encoder's unit count is increased, with no modifications to the rest of the components.

This approach to scaling allows for a controlled alteration of the models' capacity, which in turn allows for a more precise study into how such modifications influence their syntax learning capabilities.

5 RESULTS AND ANALYSIS

Before analyzing the embeddings extracted from the models to assess their capability to learn the syntax of the programming language and answer our research question, we must first ensure the reliability of the probing approach used to obtain those embeddings. Therefore, in the following, we first conduct a validation of our probing approach before proceeding to answer our research question.

5.1 Evaluation of our Probing Approach DeepCodeProbe

As described in Section 3.2, we evaluate DeepCodeProbe by first assessing the validity of the $\langle d, c, u \rangle$ tuple and the embeddings extracted from the models. As AST-NN and FuncGNN are designed specifically for CCD, we use the same dataset provided by their authors (AST-NN [87] and FuncGNN [50]) for the validation of DeepCodeProbe on these models. In the case of SummarizationTF and CodeSumDRL, the authors did not originally include a code clone dataset.

Algorithm 5 Generating $\langle d, c, u \rangle$ tuples from ASTs**Require:** Source code *code***Ensure:** Tuples of node positions (D), children count (C), and unary label (U) for AST processing

```

1: procedure TREE2TUPLE(AST)
2:   token2Index  $\leftarrow$  Load appropriate tokenizer based on the programming language
3:   maxTokenIndex  $\leftarrow$  Highest index in token2Index
4:   vocab  $\leftarrow$  token2Index
5:   function CONVERTNODETOINDEX(node)
6:     tokenIndex  $\leftarrow$  Index of node.token in vocab or maxTokenIndex if not found
7:     childIndices  $\leftarrow$  Empty list
8:     for child in node.children do
9:       childIndices.append(CONVERTNODETOINDEX(child))
10:    end for
11:    return [tokenIndex] + childIndices
12:  end function
13:  function AST2INDEX(tree)
14:    nodes  $\leftarrow$  Extract nodes from tree
15:    sequenceInfo  $\leftarrow$  Initialize empty dictionary for node info
16:    for i = 0 to length(nodes) - 1 do
17:      node  $\leftarrow$  nodes[i]
18:      nodeTokenIndex  $\leftarrow$  Index of node.type in vocab or maxTokenIndex
19:      nodeChildrenIndices  $\leftarrow$  CONVERTNODETOINDEX(node)
20:      sequenceInfo[i].d  $\leftarrow$  Extract positions for node in AST
21:      sequenceInfo[i].c  $\leftarrow$  nodeChildrenIndices
22:      sequenceInfo[i].u  $\leftarrow$  nodeTokenIndex
23:    end for
24:    D  $\leftarrow$  Extract position values from sequenceInfo for all nodes
25:    C  $\leftarrow$  Extract children count from sequenceInfo for all nodes
26:    U  $\leftarrow$  Extract type indices from sequenceInfo for all nodes
27:    return {d : D, c : C, u : U}
28:  end function
29:  return AST2INDEX(AST)
30: end procedure

```

Hence, to assess the validity of our probing approach on these models, we use the code clone dataset of AST-NN for SummarizationTF (as they are both trained on Java) and the Python code clone detection dataset from [54] which was used to train a code clone detector based on CodeBERT [16] for Python code clone detection [32], for CodeSumDRL.

Table 1 presents the average cosine similarity for the $\langle d, c, u \rangle$ tuples for each of the models under study. The “Programming Language” column specifies the programming language on which the model is trained. The “Comparison criterion” denotes which subsets of the datasets were used for the similarity comparison, with “similar” indicating comparison among code clones (Types 1 and 2) and “dissimilar” indicating comparison among non-clone pairs. Finally, the “Average Similarity” columns show the average cosine similarity across all $\langle d, c, u \rangle$ tuples constructed for each model and comparison criterion.

As shown by the results in Table 1, we can observe a significant difference in the $\langle d, c, u \rangle$ similarities between clone and non-clone code pairs across all models. Specifically, for AST-NN we can observe a 7% difference in the representations of d tuples which encode node location, a 5% difference in the representation of c tuples which encode

Table 1. Results of Cosine similarity of $\langle d, c, u \rangle$ tuples for each model

Model	Programming Language	Similarity Criterion	Average Similarity - D(%)	Average Similarity - C(%)	Average Similarity - U(%)
AST-NN	C	Similar	64	17	43
		Dissimilar	57	12	4
	Java	Similar	80	50	76
		Dissimilar	42	16	54
FuncGNN	Java	Similar	90	86	82
		Dissimilar	57	36	68
SummarizationTF	Java	Similar	75	83	74
		Dissimilar	33	60	51
CodeSumDRL	Python	Similar	0	28	35
		Dissimilar	-11	3	20

the node’s children, and a major difference of 39% between u tuples which encode node labeling information which lines up with how large ASTs are broken down into smaller STs for this model [87]. For FuncGNN, we observe noticeable differences in representations constructed for CFGs, showing a strong differentiation based on node position, connections, and labels between clone and non-clone pairs especially across the d tuples which encode the position of each node in the CFG and the c tuples which encode the connections between the nodes. We observe a similar trend for SummarizationTF, especially with a difference of 43% for the d tuple, 23% for the c tuple, and 23% for the u tuple which lines up with how ASTs of dissimilar codes have different structures as also described in Section 3.2.1. For CodeSumDRL, we observe similar results as SummarizationTF in the similarities between the $\langle d, c, u \rangle$ tuples as well as a similar difference of distinction between the encoded representation of the nodes’ positions and children. Based on these results, we can infer that our $\langle d, c, u \rangle$ tuple representation, extracted from ASTs/CFGs, holds sufficient syntactical information and can be used for probing for syntactic information on the models under study.

With the validity of the constructed $\langle d, c, u \rangle$ tuples established, we proceed to assess the validity of the information within the embeddings extracted from each model. Table 2 presents the results of embedding validation. Following a similar methodology to validating the $\langle d, c, u \rangle$ tuples representation, we compute the average cosine similarity across both clone and non-clone pairs. Additionally, the “Trained” column indicates whether the embeddings were extracted from an untrained or trained instance of the model.

The analysis of data from Table 2 reveals a significant distinction between the embeddings of clone and non-clone codes. Specifically, we observe that after training the models, the similarity of the extracted embeddings for code clones increases. Furthermore, the distance in the embeddings extracted from the models between clone and non-clone codes changes significantly post-training, as opposed to their randomly initialized stage. In more detail, for code clones, AST-NN shows a significant increase from 4.81% to 28.89% in C, and from 0.42% to 36.43% in Java, post-training. We observe an increase in the cosine similarity of non-clone codes as well, but to a lesser degree, which can be attributed to the model’s ability to identify some form of structure in the STs which we expand more on, in RQ2 (Section 5.3). We observe a similar pattern for FuncGNN with the similarity of code clones increasing from 78.46% to 86.15% and just a 1.5% increase between the embedding similarity of non-clone codes, post-training. The marginal increase for non-clone codes shows the model’s capability to recognize underlying structural patterns which we will expand on when discussing the probing results in Section 5.2. For SummarizationTF, we also observe the same pattern of average cosine similarity increasing for code clones after training the model with a lower cosine similarity for non-clone pairs. We observe a 6.4% difference between the embeddings of code clones before and after training the model and a 9.5% increase for non-clone codes. Finally, for CodeSumDRL we observe that the embedding similarity increases from 8.71%

Table 2. Results of Cosine Similarity of embeddings before and after training

Model	Programming Language	Similarity Criterion	Trained	Average Cosine Sim (%)
AST-NN	C	Similar	Y	28.89
			N	4.81
		Dissimilar	Y	20.38
			N	1.90
	Java	Similar	Y	36.43
		Dissimilar	N	0.42
FuncGNN	Java	Similar	Y	86.15
			N	78.46
		Dissimilar	Y	72.55
			N	71.05
SummarizationTF	Java	Similar	Y	59.33
			N	52.90
		Dissimilar	Y	53.47
			N	43.97
CodeSumDRL	Python	Similar	Y	19.03
			N	8.71
		Dissimilar	Y	11.60
			N	3.19

to 19.03% for code clones, and from 3.19% to 11.6% for non-clones. These results confirm the validity of the extracted embeddings from each model for probing, as they show that each model learns some information from the syntactic representations and does not memorize its training data.

By cross-analysing the data in Tables 1 and 2, we can infer the following:

- The $\langle d, c, u \rangle$ tuple, which is constructed for each model following the model’s data representation approach, encodes the syntactic information in the AST/CFG. It should be noted that the $\langle d, c, u \rangle$ tuple construction and its syntactic representation are separate from the model embeddings and the representations that the model learns from its training data.
- The models demonstrate an ability to learn some form of representation from the input AST/CFGs, enabling the differentiation between clone and non-clone code pairs even if they are not trained for CCD.

5.2 RQ1: Are models trained on syntactical representations of code capable of learning the syntax of the programming language?

Table 3 presents the results of our probing as described in Sections 4.2 and 4.3. The “Accuracy-D”, “Accuracy-C”, and “Accuracy-U” columns indicate the probe’s accuracy in predicting the d , c , and u tuples, respectively. As we can observe from the results, the models under study are incapable of representing the syntax of the programming language in their latent space. For the AST-NN model trained for CCD on C and Java, the poor accuracy scores of 8% across all syntax tuples (d , c and u) indicate an extremely limited ability to encode programming language’s syntax in its latent space. Similarly, the SummarizationTF model for code summarization struggles with syntax learning as well, achieving low accuracies of 13.83%, 14.79%, and 14.79% for the d , c , and u tuples respectively. In contrast, the CodeSumDRL model exhibits a relatively stronger syntax learning ability, with accuracies of 41.6% for d , 33.92% for c , and 28.08% for u tuples.

on the code summarization task. While not close to the results reported for LLMs in [23, 72, 77] (where the probes exhibit over 80% accuracy in retrieving information related to the programming language’s syntax), these higher scores suggest that CodeSumDRL learns some syntactic patterns from code. The most intriguing result comes from FuncGNN’s probe. As described in Section 4.1.2, FuncGNN was designed for CCD on CFGs. Remarkably, it achieves 98% accuracy in predicting the $\langle d \rangle$ tuple that encodes edge connection information in the code’s CFG. Thus, we can conclude that FuncGNN particularly focuses on the edges of the CFGs (connections between nodes) in order to detect clones between two input codes. This observation also aligns with the results reported in [50], in which the goal of FuncGNN is to predict the GED between the CFGs of two input codes. However, its performance of 43.36% and 39.26% for $\langle d \rangle$ and $\langle u \rangle$ tuples indicate that the model is incapable of representing the full syntax of the programming language.

Table 3. Result of DeepCodeProbe’s accuracy on recovering Syntactic Information

Model	Task	Programming Language	Accuracy-D(%)	Accuracy-C(%)	Accuracy-U(%)
AST-NN	CCD	C	8.65	8.63	8.65
		Java	8.33	8.09	8.65
FuncGNN	CCD	Java	43.36	98.51	39.26
SummarizationTF	Code Summarization	Java	13.83	14.79	14.79
CodeSumDRL	Code Summarization	Python	41.60	33.92	28.08

From the results of our probing, we can see that our probes are not capable of predicting the $\langle d, c, u \rangle$ tuples for their input codes for AST-NN and SummarizationTF which as described in Section 3.1, indicates that they are incapable of learning the syntax of the programming language that they are trained on. Even though in comparison to the mentioned models, CodeSumDRL’s probe achieves higher accuracies across the $\langle d, c, u \rangle$ tuples, the results show that its syntax learning capability is limited. However, as these model show good performance on the benchmarks that they were presented with and our validation shows that they do not memorize their training data, we need to investigate what representation they learn from code for achieving their task. We investigate this in RQ2 (Section 5.3).

Findings 1: By analyzing the embeddings extracted from the models under study for clone and non-clone pairs, we can observe that the models learn some information about the programming language’s syntax. However, our probe shows that none of the models under study are capable of representing the complete syntax in their latent space. This indicates that the models under study are either incapable of representing the full syntax of the programming language in their latent space due to their limited capacity (as opposed to LLMs) or do not require the full syntax to achieve their objectives.

5.3 RQ2: In cases where models trained on syntactical representations of code fail to learn the programming language’s syntax, do they learn abstract patterns based on syntax?

Our findings in RQ1 indicate that the models under study are incapable of representing the complete syntax of the programming language. However, as they are trained on syntactically valid representations of code, and as our results for cosine similarity of clone and non-clone codes across the extracted embeddings show some retention of syntax information, we aim to investigate what representation from the syntax these models learn. Furthermore, as our results for FuncGNN in RQ1 reveal what it learns and which parts of the CFGs it pays attention to for CCD, we do not further

analyze it in this RQ. Therefore, to investigate the representations learned by the models, we keep the probing approach the same while changing the $\langle d, c, u \rangle$ tuple construction as follows, to probe the models for abstractions of syntactic information:

- c : A binary flag for each node that indicates whether the node has a child in the AST or not. Here, as opposed to the approach taken in RQ1, we do not include the child information for each node to be re-constructed and instead only investigate whether the models under study are capable of learning whether a node in the AST has children or not.
- u : The direct label of the node extracted from the AST. As opposed to the original u tuple which was extracted from the vector representation used by each model.
- d : We have decided to discard this vector representation since the new C and U tuples, are insufficient for reconstructing the AST. As such, encoding the position information is no longer helpful.

Listing 2 shows the $\langle c, u \rangle$ tuple constructed from the running example presented in Figure 2.

Table 4. Result of DeepCodeProbe’s accuracy on recovering Syntactic Information

Model	Task	Programming Language	Accuracy-C(%)	Accuracy-U(%)
AST-NN	CCD	C	99.17	62.11
		Java	97.50	61.25
SummarizationTF	Code Summarization	Python	73.64	60.97
CodeSumDRL	Code Summarization	Python	43.21	32.03

Table 4 shows the results of our probing approach with the new $\langle c, u \rangle$ tuple. By designing a more abstract data representation, the accuracy of the probe improves significantly compared to probing for syntax for AST-NN and SummarizationTF while only a relatively small improvement is observed for CodeSumDRL. This is an important finding as it demonstrates that, for software maintenance tasks, selecting a task-specific data representation that aligns with the models’ capabilities eliminates the necessity for the model to fully learn the programming language syntax; an abstraction of it suffices.

Conversely, the slight enhancement observed in CodeSumDRL’s probe results can be attributed to two factors:

- Even though CodeSumDRL uses ASTs to represent code, in contrast to AST-NN and SummarizationTF, it follows a multi-step process to encode information (an LSTM for processing the natural language descriptions and

```

1  c = [
2      1 # node 1 has children.
3      1 # node 2 has children,
4      1 # node 3 has children,
5      .....
6      0 # node 8 has no children
7      0 # node 9 has no children
8      .....
9  ]
10 u = [Module, FunctionDef, Expr, Arguments, ....]
```

Listing 2. Generated $\langle c, u \rangle$ tuple for the AST in Figure 2

a Tree-based-RNN for processing the AST) as described in Section 4.1.4. As such, we will further investigate CodeSumDRL’s syntax learning capabilities in RQ3 (Section 5.4).

- The complex data representation scheme which abstracts many of the details from the AST alongside the limited capacity of the model, results in loss of syntactic information.

Despite CodeSumDRL’s limited capacity to represent detailed and abstract syntactic information, it achieves commendable results on established code summarization benchmarks [78]. The performance of CodeSumDRL on code summarization benchmarks alongside the high level of abstract syntax representation retention on AST-NN and SummarizationTF, indicate the importance of using explicit information encoding for small models. This observation is particularly relevant given our choice of models to study. As described in Section 4.1, we chose models that are trained not on raw code (which contains syntax information implicitly) but on the syntactically valid representation of code (which contains the syntax explicitly). By cross-analyzing the results of RQ1 and RQ2 we can infer the following:

- For smaller ML models, using representations that explicitly encode the syntax of the programming language, allows for some level of syntax learning capability regardless of the task and models’ capacity.
- AST and CFG representations of code exclude redundant details such as comments, docstrings, and variable names, resulting in a significantly reduced representation space compared to using raw code text as input for models. This streamlined representation space is advantageous as it allows models to learn abstractions of the programming language’s syntax even when the models are not large (i.e. have a high number of learnable parameters).

Findings 2: By using a more abstract data representation for probing the models, we can observe a significant increase in the syntactic information captured in their latent space, except for CodeSumDRL which uses complex representations of ASTs. For AST-NN and SummarizationTF, we can observe that they are capable of retaining information about structural information and the relationship between nodes in an AST to accomplish their tasks without the need to learn the entire syntax of the programming language.

5.4 RQ3: Does increasing the capacity of models without changing their architecture improve syntax learning capabilities?

The findings from RQ1 and RQ2 suggest that the models studied do not fully capture the syntax of the programming language within their latent space. Instead, they tend to abstract the syntax to some degree. To explore this further, we investigate whether increasing each model’s capacity—specifically, by increasing the number of parameters in the layers responsible for extracting embeddings (as outlined in Section 4.4)—can enhance their learning capabilities. This investigation allows us to discern whether the models’ limitation in representing full syntax stems from insufficient capacity or from inherent architectural constraints.

Table 5 presents the results of our experiments with the “Size” column indicating the number of parameters for each model. We use the same $\langle d, c, u \rangle$ tuple construction and probing approach from RQ1 to probe the models under study. Analysing the results presented in Table 5, we can observe that:

- AST-NN: For both C and Java, when we scale up from the original 128 parameters to 256, we obtain an increase in the accuracy rates for d, c and u predictions. However, as we expand the model beyond this point, the rate of

Table 5. Result of DeepCodeProbe’s $\langle d, c, u \rangle$ accuracy on models with higher capacity

Model	Size	Programming Language	Accuracy-D(%)	Accuracy-C(%)	Accuracy-U(%)
AST-NN	256	C	13.11	15.02	15.40
		Java	10.47	9.30	8.92
	512	C	8.62	8.73	8.73
		Java	8.94	8.82	8.95
FuncGNN	1024	Java	44.24	98.60	39.90
SummarizationTF	1024	Java	13.82	13.81	13.74
	2048		13.79	13.60	13.44
CodeSumDRL	1024	Python	50.41	42.75	36.72
	2048		50.23	42.97	36.30
	4096		43.67	40.22	34.31

improvement in accuracy diminishes. This suggests that the model’s capacity to capture syntactic information does not proportionally increase with size, given the architecture in place.

- FuncGNN: Scaling the number of parameters for FuncGNN from 512 to 1024, results in no noticeable difference in the accuracy of retrieving the $\langle d, c, u \rangle$ tuples. Considering the results obtained in RQ1, where we observed that FuncGNN focuses on the edges of CFGs, by scaling up the model we do not observe any differences in the model’s capability to discern between node position and node types for clone detection. Furthermore, we observe a decrease in the model’s capability in CCD indicating overfitting, meaning that the model has reached its optimal capacity for the current architecture.
- SummarizationTF: Scaling the number of parameters from 512 to 1024, displays no improvement in increasing the accuracy of retrieving the d tuple and a decrease in the probe’s capability to predict the c and u tuples. This suggests that the model has reached its learning capacity within the current architectural constraints and scaling it up further will only result in overfitting and diminishing returns.
- CodeSumDRL: Increasing the number of parameters from 512 to 1024 shows a noticeable improvement in the probe’s accuracy in predicting the $\langle d, c, u \rangle$ tuples. Further scaling to 2048 parameters does not result in more improvement. However, no overfitting is observed. By further scaling to 4096 parameters, there’s a drop in the probe’s accuracy in predicting the $\langle d, c, u \rangle$ tuples compared to 2048 parameters, suggesting that the model has reached its optimal capacity for the current architecture, leading to overfitting in learning syntactic representations.

Given the results of scaling the models, and given the architecture of each model as described in section 4.1, we can observe that increasing the models’ size provides diminishing returns for each model after a certain point. Consistent with existing literature [34], each successive layer in a DNN learns more complex representations from the previous layers’ outputs. Our results also show that beyond a certain size and without changing the models’ architecture, their capacity to learn syntactic information stops improving and shows a decline, indicating potential overfitting. Moreover, the models’ effectiveness on their original tasks shows little enhancement with increased size, suggesting that the initial parameter counts were already optimal and beneficial for maintaining the model’s generalization capabilities. These observations, coupled with the results obtained from probing the original models on their syntax learning ability in the previous RQs lead us to an important conclusion: *if the task and data representations are selected adequately, there is no need to employ models that learn the full syntax of the programming language for software maintenance.*

Findings 3: Scaling models trained on code, provides marginal returns on their syntax learning capabilities without showing much improvement on the models’ original tasks. Given that the models are trained on syntactical representations of code, and their much smaller size compared to LLMs, our results show that there is no need to have the models learn the full syntactic rules of the programming language as long the task and data representation are selected adequately.

6 RELATED WORKS

Given the success of BERT [15], researchers have conducted numerous studies on how BERT and BERT-based models learn and represent data for downstream tasks [57]. Although these studies primarily focus on general linguistic features rather than syntax-specific to code, the methodologies developed have been extensively applied to probe larger transformer-based models on how they learn and how they represent data in their latent space. Liu et al. [38] investigated the linguistic capabilities of pretrained models like ELMo, OpenAI’s transformer, and BERT through seventeen probing tasks. They found that while these models perform well on most tasks, they struggle with complex linguistic challenges. Hewitt and Manning [24] introduced a structural probe to assess whether complete syntax trees are embedded within the linear transformations of neural network word representations (i.e. embeddings). They showed that models like ELMo and BERT encode syntax trees within their vector geometries. Miaschi et al. [46] compared the linguistic knowledge in the BERT’s embeddings and Word2Vec. They used probing tasks targeting various sentence-level features and found that BERT’s aggregated sentence representations hold comparable linguistic knowledge to Word2Vec.

With the advent of LLMs and their emerging capabilities, there is increasing interest in understanding how these models process information and make decisions based on their inputs. In the context of software engineering, researchers have adapted probing techniques originally used to understand the linguistic features that models learn from their training data to examine what models trained on code learn from their respective datasets. This has led to the creation of novel probing tasks specifically for LLMs trained on code. Wan et al. [77] have probed pre-trained language models trained on code to investigate if they learn meaningful abstractions related to programming constructs, control structures, and data types. They conducted structural analyses from three distinct perspectives: attention analysis, probing on the word embeddings, and syntax tree induction. Their findings indicate that attention strongly aligns with the syntax structure of code, and that pre-trained models can preserve this syntax structure in their intermediate representations. Similarly, Troshin et al. [72] evaluated the capability of pre-trained code models, such as CodeBERT [16] and CodeT5 [80], in understanding source code, through diagnostic probing tasks, moving beyond task-specific models for code processing such as code generation and summarization. They introduced a variety of probing tasks to assess models’ understanding of both syntactic structures and semantic properties of code, including namespaces, data flow, and semantic equivalence. Karmakar et al. [29], investigated the extent to which pre-trained code models encode various characteristics of source code by designing and employing four diagnostic probes focusing on surface-level, syntactic, structural, and semantic information of code. Their methodology involves probing BERT [15], CodeBERT [16], CodeBERTa [83], and GraphCodeBERT [20] with tasks aimed at understanding the extent to which these models capture code-specific properties. Their findings show that these models can indeed capture the syntactic and semantic layers of code with varying effectiveness. However, all the models studied struggle to accurately comprehend the structural complexity of code, particularly the interconnected paths within it.

7 DISCUSSION

Based on the numerous experiments we conducted while replicating the results of the models under study, as described in Section 4.1, along with the results of our probing, we identify best practices for training models for software maintenance on syntactic representations of code. We expand on each of these practices as follows.

7.1 Efficacy of Syntactical Representations in Model Performance

Our experiment results lead us to an important conclusion: models do not need to fully learn the syntax of the programming language to perform effectively on software maintenance tasks. Our analyses show that deep learning models can extract and learn abstractions of the syntax, which can be sufficient for achieving their objectives. This ability to learn abstractions stems from training on syntactically valid representations of code, rather than treating code merely as text. Traditionally, approaches for code clone detection and code summarization have trained models on code by treating code as text [85]. However, our probing reveals that leveraging artifacts extracted from code, which remove task-irrelevant information, allows us to train smaller yet effective models. These models can learn abstractions of the programming language’s syntax without sacrificing performance or inference time. Empirical studies indicate that the performance of deployed ML models is often constrained during inference, as large models require extensive processing resources and time [10, 85, 88]. By adopting syntactically informed representations for code, we can enhance model efficiency and significantly reduce model size (compared to LLMs) without compromising predictive performance on specific downstream tasks.

7.2 Tailoring Data Representations to Task-Specific Needs

For each of the models under study, the training data consists of either AST/CFG (FuncGNN, SummarizationTF, CodeSumDRL) or an abstraction of it (AST-NN). However, as described in Section 4.1 each model follows a different data representation scheme to further break down the complexities of AST/CFG. AST-NN breaks down the ASTs into smaller STs in order to solve the problem of overly large inputs. It also uses Word2Vec representation of node labels instead of simply going over all the unique tokens and assigning them an index. This representation allows AST-NN to achieve good performance on CCD while maintaining a compact size. In contrast, FuncGNN uses statement-level tokenization of inputs instead of word-level for representing the nodes in the CFG, resulting in a smaller search space for learning the labels of each node and enabling the model to capture finer details in CFGs to achieve CCD, as shown in Section 5.2. SummarizationTF and CodeSumDRL are models trained to provide natural language descriptions of code. This can be viewed as a translation task (by looking at code as one language and the natural language as the target language). In fact, most comment generation and code summarization approaches are conceptualized as language translation tasks in which each block of code is systematically mapped to a corresponding natural language description [85]. However, as described in Section 4.1.3 and Section 4.1.4, using AST extracted from code rather than treating code as text allows SummarizationTF and CodeSumDRL to achieve good performance while being much smaller than LLMs and still capable of learning abstractions from the programming language’s syntax. Therefore, we recommend that for tasks requiring models to be trained on code, practitioners should use syntactic representations such as AST or CFG instead of treating code as text.

7.3 Enhancing Model Reliability through Interpretability Probes

To have reliable models, an important aspect to consider is interpretability. Exploring models' behavior through probing provides insights into their decision-making. Probing helps developers pinpoint errors, refine the model with additional training data, and improve interpretability. DeepCodeProbe displays the benefits of probing by showing how it allows understanding what the models focus on to achieve their tasks. Similar approaches allow developers to investigate specific instances where models under study make errors, facilitating a better understanding of the reasons behind such mistakes.

With the rise of LLMs, there is a tendency to train models on extensive corpora of source code, which enables these models to implicitly learn the syntax of programming languages. This approach has shown success, as various probing studies on LLMs report that these models can represent programming language syntax in their latent space [23, 29, 72, 77]. However, LLMs are expensive to train [62], require enormous amounts of training data [91], are prone to hallucinations [89], and lack interpretability [90]. In contrast, the models we have studied are significantly smaller and less resource-intensive. Their training is computationally inexpensive compared to LLMs, and they can be trained on readily available data without complex preprocessing. Consequently, inference on these models is also inexpensive. They are all based on either RNNs, encoder-decoder, or seq2seq architectures, which are well-studied in ML and SE literature for interpretability [12]. As our study results demonstrate, these models can learn abstractions from the syntax of programming languages to achieve their tasks effectively.

While LLMs have shown impressive capabilities in code generation and understanding, relying solely on their outputs can be problematic, especially in safety-critical domains like software maintenance. The lack of interpretability and potential for hallucinations in LLMs can introduce bugs or vulnerabilities, making them unreliable for such critical tasks [69]. In contrast, the smaller models we studied offer advantages such as lower computational requirements, well-understood architectures, and higher confidence in their decision-making processes and learned representations. Therefore, for software maintenance tasks, developers may find it beneficial to consider using smaller, more interpretable models rather than LLMs, given the issues described above.

8 THREATS TO VALIDITY

Threats to **internal validity** concern factors, internal to our study, that could have influenced our results. Our data representation scheme for each model which is built upon AST-Probe, can be considered a potential threat to the validity of our research. Probing has been used in other contexts in NLP for uncovering what the models learn, and our results show that by extracting a syntactically valid vector representation, we can probe models for syntactical information. Additionally, our data transformation and embedding extraction for each of the models under study, may not encompass all the required steps to understand the internal representations of DL models. We mitigate this threat by experimenting with multiple data representation approaches and extracting the embeddings from different layers of the models under study to have a comprehensive understanding of the probing task and the obtained results.

Threats to **construct validity** concern the relationship between theory and observation. The primary threat to the construct validity of our study is the probing technique we propose. This technique could potentially provide a distorted view of what models have truly learned, thereby impacting our analysis and the conclusions drawn from it. In order to mitigate this threat, we validate the extracted embeddings by comparing the cosine similarity of code clone and non-clone pairs from models trained on syntactically valid code representations. Furthermore, we establish the probe's validity by demonstrating that it does not learn new representations but reveals pre-existing information in the model's

latent space. Moreover, we have conducted thorough experiments using embeddings from both trained and randomly initialized models, ensuring our probing approach robustly reflects the underlying syntactic learning of the models.

Threats to **conclusion validity** concern the relationship between theory and outcome and are mainly related to our analysis done on the our probes' results. To mitigate this threat we have conducted probing on models with the same architecture but higher capacities to ensure that our conclusions and the suggested guidelines are based on both theoretical and empirical evidence.

Threats to **external validity** concern the generalization of our findings. We have selected models for software maintenance across two different tasks with different data representation schemes and trained on different programming languages, in order to have a comprehensive analysis and mitigate this threat. Furthermore, there exists the possibility that our probing approach produces different results than the ones reported on other models with relatively the same capacity but different architecture and data processing approaches. As there exists a plethora of ML models for software maintenance tasks, we have selected the models under study in a manner to cover most of the design choices that are being utilized (encoder-decoder models, graph neural networks, and seq2seq models). It should be noted that in order to be able to validate and analyze the results of our proposed probing approach, we were required to study models that had made both their code and data available and were trained on syntactically correct code, to replicate their results and ensure that any negative results and analysis were not resulted from incorrect replication. Given such limitations, the models under study cover the majority of the proposed approaches across programming languages and architectural designs. Given that our data transformation and embedding extraction are model agnostic, we believe that our probing approach can be used to probe other models as well.

Threats to **reliability and validity** concern the possibility of replicating this study. We have provided all the necessary details needed for replication, sharing our full replication package [42]. Additionally, since our probing approach requires re-training the models under study, our replication package includes the code, links to data repositories, and the configurations for re-training the models.

9 CONCLUSION

In this study, we introduced DeepCodeProbe , a novel probing approach for assessing the syntax learning capabilities of non-transformer-based, ML models trained on code for software maintenance tasks. Our probing approach is based on syntactically valid constructs extracted from codes which allows for efficient and reliable probing of ML models on their syntax learning capabilities alongside providing interpretability on the representations learned by the models to achieve their tasks. Our results show that while non-transformer-based, ML models are unable to fully capture the complete set of syntactic rules of programming languages, they are capable of learning relevant abstractions and patterns within the language syntax. This suggests that such models, despite their relatively simple and small architectures compared to LLMs, can still acquire meaningful syntactic knowledge from code-based training data. Furthermore, our experiments demonstrate that simply increasing the capacity of these models, without changing their architecture, leads to only marginal improvements in their full syntax learning abilities. These results indicate that the architectural design of the model plays a crucial role in determining its syntax learning capabilities, beyond just the amount of training data or model size. These findings have important implications for the development of small, efficient ML models for code-related tasks, as they highlight the need to carefully consider model architecture choices in addition to scaling up model size. Finally, we provide a set of guidelines based on our proposed probing approach and the subsequent conducted analysis on the models under study for training ML models for software maintenance.

To allow for the development and deployment of more reliable ML models for software maintenance, we aim to expand DeepCodeProbe into a framework that can be deployed on top of trained models. Doing so would enable developers and researchers to systematically evaluate and enhance the syntax learning capabilities of ML models, ensuring their effectiveness in software maintenance tasks.

10 ACKNOWLEDGMENTS

This work is partially supported by the Fonds de Recherche du Quebec (FRQ), the Canadian Institute for Advanced Research (CIFAR), and the Natural Sciences and Engineering Research Council of Canada (NSERC).

REFERENCES

- [1] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2020. A transformer-based approach for source code summarization. *arXiv preprint arXiv:2005.00653* (2020).
- [2] Qurat Ul Ain, Wasi Haider Butt, Muhammad Waseem Anwar, Farooque Azam, and Bilal Maqbool. 2019. A systematic review on code clone detection. *IEEE access* 7 (2019), 86121–86144.
- [3] Miltiadis Allamanis, Hao Peng, and Charles Sutton. 2016. A convolutional attention network for extreme summarization of source code. In *International conference on machine learning*. PMLR, 2091–2100.
- [4] Maryam Ashoori and Justin D Weisz. 2019. In AI we trust? Factors that influence trustworthiness of AI-infused decision-making processes. *arXiv preprint arXiv:1912.02675* (2019).
- [5] Antoine Barbez, Foutse Khomh, and Yann-Gaël Guéhéneuc. 2019. Deep learning anti-patterns from code metrics history. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 114–124.
- [6] Yonatan Belinkov. 2022. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics* 48, 1 (2022), 207–219.
- [7] Hans Christian Benestad, Bente Anda, and Erik Arisholm. 2010. Understanding cost drivers of software evolution: a quantitative and qualitative investigation of change effort in two evolving software systems. *Empirical Software Engineering* 15 (2010), 166–203.
- [8] Sahil Bhatia, Pushmeet Kohli, and Rishabh Singh. 2018. Neuro-symbolic program corrector for introductory programming assignments. In *Proceedings of the 40th International Conference on Software Engineering*. 60–70.
- [9] Diogo V Carvalho, Eduardo M Pereira, and Jaime S Cardoso. 2019. Machine learning interpretability: A survey on methods and metrics. *Electronics* 8, 8 (2019), 832.
- [10] Zhenpeng Chen, Yanbin Cao, Yuanqiang Liu, Haoyu Wang, Tao Xie, and Xuanzhe Liu. 2020. A comprehensive study on challenges in deploying deep learning based software. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 750–762.
- [11] Michel Chlowlowicz, Etienne Duris, and Gilles Roussel. 2009. Syntax tree fingerprinting for source code similarity detection. In *2009 IEEE 17th international conference on program comprehension*. IEEE, 243–247.
- [12] Shivani Choudhary, Niladri Chatterjee, and Subir Kumar Saha. 2022. Interpretation of black box nlp models: A survey. *arXiv preprint arXiv:2203.17081* (2022).
- [13] Hoa Khanh Dam, Trang Pham, Shien Wee Ng, Truyen Tran, John Grundy, Aditya Ghose, Taeksu Kim, and Chul-Joo Kim. 2018. A deep tree-based model for software defect prediction. *arXiv preprint arXiv:1802.00921* (2018).
- [14] Greg d'Eon, Jason d'Eon, James R Wright, and Kevin Leyton-Brown. 2022. The spotlight: A general method for discovering systematic errors in deep learning models. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*. 1962–1981.
- [15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [16] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
- [17] Yi Gao, Zan Wang, Shuang Liu, Lin Yang, Wei Sang, and Yuanfang Cai. 2019. TECCD: A tree embedding approach for code clone detection. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 145–156.
- [18] Emily Goodwin, Koustuv Sinha, and Timothy J O'Donnell. 2020. Probing linguistic systematicity. *arXiv preprint arXiv:2005.04315* (2020).
- [19] David Gros, Hariharan Sezhiyan, Prem Devanbu, and Zhou Yu. 2020. Code to comment" translation" data, metrics, baselining & evaluation. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 746–757.
- [20] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366* (2020).
- [21] Rahul Gupta, Soham Pal, Aditya Kanade, and Shirish Shevade. 2017. Deepfix: Fixing common c language errors by deep learning. In *Proceedings of the aaai conference on artificial intelligence*, Vol. 31.

- [22] Patrick Hall, Navdeep Gill, and Nicholas Schmidt. 2019. Proposed guidelines for the responsible use of explainable machine learning. *arXiv preprint arXiv:1906.03533* (2019).
- [23] José Antonio Hernández López, Martin Weyssow, Jesús Sánchez Cuadrado, and Houari Sahraoui. 2022. AST-Probe: Recovering abstract syntax trees from hidden representations of pre-trained language models. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–11.
- [24] John Hewitt and Christopher D Manning. 2019. A structural probe for finding syntax in word representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 4129–4138.
- [25] Xing Hu, Ge Li, Xin Xia, David Lo, and Zhi Jin. 2018. Deep code comment generation. In *Proceedings of the 26th conference on program comprehension*. 200–210.
- [26] Wei Hua, Yulei Sui, Yao Wan, Guangzhong Liu, and Guandong Xu. 2020. Fcca: Hybrid code representation for functional clone detection using attention networks. *IEEE Transactions on Reliability* 70, 1 (2020), 304–318.
- [27] Iain M Johnstone and D Michael Titterton. 2009. Statistical challenges of high-dimensional data. , 4237–4253 pages.
- [28] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [29] Anjan Karmakar and Romain Robbes. 2021. What do pre-trained code models know about code?. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1332–1336.
- [30] Stefan Koch. 2005. Evolution of open source software systems—a large-scale investigation. In *Proceedings of the 1st International Conference on Open Source Systems*. 148–153.
- [31] Lov Kumar and Santanu Ku Rath. 2016. Hybrid functional link artificial neural network approach for predicting maintainability of object-oriented software. *Journal of Systems and Software* 121 (2016), 170–190.
- [32] LazyHope. 2022. Python Programming Language Clone Detection. <https://huggingface.co/Lazyhope/python-clone-detection>
- [33] Alexander LeClair, Sakib Haque, Lingfei Wu, and Collin McMillan. 2020. Improved code summarization via a graph neural network. In *Proceedings of the 28th international conference on program comprehension*. 184–195.
- [34] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *nature* 521, 7553 (2015), 436–444.
- [35] Maggie Lei, Hao Li, Ji Li, Namrata Aundhkar, and Dae-Kyoo Kim. 2022. Deep learning application on code clone detection: A review of current knowledge. *Journal of Systems and Software* 184 (2022), 111141.
- [36] Liuqing Li, He Feng, Wenjie Zhuang, Na Meng, and Barbara Ryder. 2017. Cclerner: A deep learning-based clone detection approach. In *2017 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 249–260.
- [37] Zheng Li, Yonghao Wu, Bin Peng, Xiang Chen, Zeyu Sun, Yong Liu, and Doyle Paul. 2022. Setransformer: A transformer-based code semantic parser for code comment generation. *IEEE Transactions on Reliability* 72, 1 (2022), 258–273.
- [38] Nelson F Liu, Matt Gardner, Yonatan Belinkov, Matthew E Peters, and Noah A Smith. 2019. Linguistic knowledge and transferability of contextual representations. *arXiv preprint arXiv:1903.08855* (2019).
- [39] Yibin Liu, Yanhui Li, Jianbo Guo, Yuming Zhou, and Baowen Xu. 2018. Connecting software metrics across versions to predict defects. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 232–243.
- [40] Siwen Luo, Hamish Ivison, Soyeon Caren Han, and Josiah Poon. 2021. Local interpretations for explainable natural language processing: A survey. *Comput. Surveys* (2021).
- [41] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. Coconut: combining context-aware neural translation models using ensemble for program repair. In *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*. 101–114.
- [42] Vahid Majdinasab. 2024. DeepCodeProbe. <https://github.com/CommissarSilver/DeepCodeProbe>.
- [43] Vahid Majdinasab, Michael Joshua Bishop, Shawn Rasheed, Arghavan Moradidakhel, Amjed Tahir, and Foutse Khomh. 2023. Assessing the Security of GitHub Copilot Generated Code—A Targeted Replication Study. *arXiv preprint arXiv:2311.11177* (2023).
- [44] Rowan Hall Maudslay, Josef Valvoda, Tiago Pimentel, Adina Williams, and Ryan Cotterell. 2020. A tale of a probe and a parser. *arXiv preprint arXiv:2005.01641* (2020).
- [45] Tom Mens, Michel Wermelinger, Stéphane Ducasse, Serge Demeyer, Robert Hirschfeld, and Mehdi Jazayeri. 2005. Challenges in software evolution. In *Eighth International Workshop on Principles of Software Evolution (IWPSE'05)*. IEEE, 13–22.
- [46] Alessio Miaschi and Felice Dell’Orletta. 2020. Contextual and non-contextual word embeddings: an in-depth linguistic investigation. In *Proceedings of the 5th Workshop on Representation Learning for NLP*. 110–119.
- [47] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems* 26 (2013).
- [48] Ruslan Mitkov, Richard Evans, Constantin Orăsan, Iustin Dornescu, and Miguel Rios. 2012. Coreference resolution: To what extent does it help NLP applications?. In *Text, Speech and Dialogue: 15th International Conference, TSD 2012, Brno, Czech Republic, September 3-7, 2012. Proceedings 15*. Springer, 16–27.
- [49] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. 2016. Convolutional neural networks over tree structures for programming language processing. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 30.

- [50] Aravind Nair, Avijit Roy, and Karl Meinke. 2020. funcgnn: A graph neural network approach to program similarity. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–11.
- [51] Sydney Nguyen, Hannah McLean Babe, Yangtian Zi, Arjun Guha, Carolyn Jane Anderson, and Molly Q Feldman. 2024. How Beginning Programmers and Code LLMs (Mis) read Each Other. *arXiv preprint arXiv:2401.15232* (2024).
- [52] Rangeet Pan, Ali Reza Ibrahimzade, Rahul Krishna, Divya Sankar, Lambert Pougum Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [53] Tiago Pimentel, Josef Valvoda, Rowan Hall Maudslay, Ran Zmigrod, Adina Williams, and Ryan Cotterell. 2020. Information-theoretic probing for linguistic structure. *arXiv preprint arXiv:2004.03061* (2020).
- [54] PoolC. [n. d.]. PoolC/1-fold-clone-detection-600k-5fold. Datasets at Hugging face. <https://huggingface.co/datasets/PoolC/1-fold-clone-detection-600k-5fold>
- [55] Václav Rajlich. 2014. Software evolution and maintenance. In *Future of Software Engineering Proceedings*. 133–144.
- [56] Sanka Rasnayaka, Guanlin Wang, Ridwan Shariffdeen, and Ganesh Neelakanta Iyer. 2024. An empirical study on usage and perceptions of llms in a software engineering project. *arXiv preprint arXiv:2401.16186* (2024).
- [57] Anna Rogers, Olga Kovaleva, and Anna Rumshisky. 2021. A primer in BERTology: What we know about how BERT works. *Transactions of the Association for Computational Linguistics* 8 (2021), 842–866.
- [58] Chanchal Kumar Roy and James R Cordy. 2007. A survey on software clone detection research. *Queen’s School of computing TR* 541, 115 (2007), 64–68.
- [59] Chanchal K Roy, James R Cordy, and Rainer Koschke. 2009. Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Science of computer programming* 74, 7 (2009), 470–495.
- [60] Cynthia Rudin. 2019. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature machine intelligence* 1, 5 (2019), 206–215.
- [61] David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. 2015. Hidden technical debt in machine learning systems. *Advances in neural information processing systems* 28 (2015).
- [62] Or Sharir, Barak Peleg, and Yoav Shoham. 2020. The cost of training nlp models: A concise overview. *arXiv preprint arXiv:2004.08900* (2020).
- [63] Yusuke Shido, Yasuaki Kobayashi, Akihiro Yamamoto, Atsushi Miyamoto, and Tadayuki Matsumura. 2019. Automatic source code summarization with extended tree-lstm. In *2019 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.
- [64] Nimit Sohoni, Jared Dunnmon, Geoffrey Angus, Albert Gu, and Christopher Ré. 2020. No subclass left behind: Fine-grained robustness in coarse-grained classification problems. *Advances in Neural Information Processing Systems* 33 (2020), 19339–19352.
- [65] Xiaotao Song, Hailong Sun, Xu Wang, and Jiafei Yan. 2019. A survey of automatic generation of source code comments: Algorithms and techniques. *IEEE Access* 7 (2019), 111411–111428.
- [66] Jan Svacina, Jonathan Simmons, and Tomas Cerny. 2020. Semantic code clone detection for enterprise applications. In *Proceedings of the 35th Annual ACM Symposium on Applied Computing*. 129–131.
- [67] Jeffrey Svajlenko, Judith F Islam, Iman Keivanloo, Chanchal K Roy, and Mohammad Mamun Mia. 2014. Towards a big data curated benchmark of inter-project code clones. In *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 476–480.
- [68] Kai Sheng Tai, Richard Socher, and Christopher D Manning. 2015. Improved semantic representations from tree-structured long short-term memory networks. *arXiv preprint arXiv:1503.00075* (2015).
- [69] Florian Tambo, Arghavan Moradi Dakhel, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Giuliano Antoniol. 2024. Bugs in large language models generated code. *arXiv preprint arXiv:2403.08937* (2024).
- [70] Ningzhi Tang, Meng Chen, Zheng Ning, Aakash Bansal, Yu Huang, Collin McMillan, and Toby Jia-Jun Li. 2024. A Study on Developer Behaviors for Validating and Repairing LLM-Generated Code Using Eye Tracking and IDE Actions. *arXiv preprint arXiv:2405.16081* (2024).
- [71] Haonan Tong, Bin Liu, and Shihai Wang. 2018. Software defect prediction using stacked denoising autoencoders and two-stage ensemble learning. *Information and Software Technology* 96 (2018), 94–111.
- [72] Sergey Troshin and Nadezhda Chirkova. 2022. Probing pretrained models of source code. *arXiv preprint arXiv:2202.08975* (2022).
- [73] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2018. Deep learning similarities from different representations of source code. In *Proceedings of the 15th international conference on mining software repositories*. 542–553.
- [74] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An empirical study on learning bug-fixing patches in the wild via neural machine translation. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 28, 4 (2019), 1–29.
- [75] Robin Vandaele, Bo Kang, Tijl De Bie, and Yvan Saeys. 2022. The Curse Revisited: When are Distances Informative for the Ground Truth in Noisy High-Dimensional Data?. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 2158–2172.
- [76] Giulia Vilone and Luca Longo. 2020. Explainable artificial intelligence: a systematic review. *arXiv preprint arXiv:2006.00093* (2020).
- [77] Yao Wan, Wei Zhao, Hongyu Zhang, Yulei Sui, Guandong Xu, and Hai Jin. 2022. What do they capture? a structural analysis of pre-trained language models for source code. In *Proceedings of the 44th International Conference on Software Engineering*. 2377–2388.
- [78] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S Yu. 2018. Improving automatic source code summarization via deep reinforcement learning. In *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*. 397–407.

- [79] Wei Wang, Huilong Ning, Gaowei Zhang, Libo Liu, and Yi Wang. 2024. Rocks Coding, Not Development—A Human-Centric, Experimental Evaluation of LLM-Supported SE Tasks. *arXiv preprint arXiv:2402.05650* (2024).
- [80] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021).
- [81] Bolin Wei, Ge Li, Xin Xia, Zhiyi Fu, and Zhi Jin. 2019. Code generation as a dual task of code summarization. *Advances in neural information processing systems* 32 (2019).
- [82] Martin White, Michele Tufano, Christopher Vendome, and Denys Poshyvanyk. 2016. Deep learning code fragments for code clone detection. In *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*. 87–98.
- [83] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2019. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771* (2019).
- [84] Yadollah Yaghoobzadeh, Katharina Kann, Timothy J Hazen, Eneko Agirre, and Hinrich Schütze. 2019. Probing for semantic classes: Diagnosing the meaning content of word embeddings. *arXiv preprint arXiv:1906.03608* (2019).
- [85] Yanming Yang, Xin Xia, David Lo, and John Grundy. 2022. A survey on deep learning for software engineering. *ACM Computing Surveys (CSUR)* 54, 10s (2022), 1–73.
- [86] Chunyan Zhang, Junchao Wang, Qinglei Zhou, Ting Xu, Ke Tang, Hairan Gui, and Fudong Liu. 2022. A survey of automatic source code summarization. *Symmetry* 14, 3 (2022), 471.
- [87] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, Kaixuan Wang, and Xudong Liu. 2019. A novel neural source code representation based on abstract syntax tree. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 783–794.
- [88] Tianyi Zhang, Cuiyun Gao, Lei Ma, Michael Lyu, and Miryung Kim. 2019. An empirical study of common challenges in developing deep learning applications. In *2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 104–115.
- [89] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, et al. 2023. Siren’s song in the AI ocean: a survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219* (2023).
- [90] Haiyan Zhao, Hanjie Chen, Fan Yang, Ninghao Liu, Huiqi Deng, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, and Mengnan Du. 2024. Explainability for large language models: A survey. *ACM Transactions on Intelligent Systems and Technology* 15, 2 (2024), 1–38.
- [91] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).
- [92] Xingyu Zhao, Alec Banks, James Sharp, Valentin Robu, David Flynn, Michael Fisher, and Xiaowei Huang. 2020. A safety framework for critical systems utilising deep neural networks. In *Computer Safety, Reliability, and Security: 39th International Conference, SAFECOMP 2020, Lisbon, Portugal, September 16–18, 2020, Proceedings* 39. Springer, 244–259.
- [93] Xiyu Zhou, Peng Liang, Beiqi Zhang, Zengyang Li, Aakash Ahmad, Mojtaba Shahin, and Muhammad Waseem. 2023. On the Concerns of Developers When Using GitHub Copilot. *arXiv preprint arXiv:2311.01020* (2023).

Received July 2024